```
getUserDataForFred = getUserData 'fred'

retryForFiveSeconds = (retryFor (seconds 5), (seconds 1))
retryForFiveSeconds getUserDataForFred, (error, data) ->
  console.log data
```

If the database finally responds after three attempts, then the console output will reflect that:

```
Error: Retry in 1000
Error: Retry in 1000
Error: Retry in 1000
Success
```

With *recursion* you can solve complicated problems with a *small amount of code*. Unfortunately, there's no such thing as a free lunch. A programming language that supports recursive functions must deal with the nature of recursion—it looks infinite.

Infinite recursion, tail calls, and the stack

Programs that make heavy use of recursion have a problem in JavaScript and CoffeeScript—they can run out of memory. Why? When you invoke a function stack, memory is allocated:

```
memoryEater = ->
memoryEater()
```

Memory is allocated for the function.

Only once the function returns is the memory allocated for it reclaimed. That's a problem with a recursive function because, before returning, a recursive function invokes itself. You can see this in action on the CoffeeScript REPL, which uses the V8 runtime. This will warn you when you've exceeded the maximum call stack size:

```
memoryEater = -> memoryEater()
memoryEater()
# RangeError: Maximum call stack size exceeded
```

The `memoryEater` function is *infinitely recursive* because it *always* invokes itself. An infinitely recursive function will cause some runtimes to